Coding Project: Stage 2 (References, Exceptions, and Subtyping)

Contents

2.1 Overview

In this stage of the project, you must implement a program that performs type checking for source code written in a simple statically typed functional fragment of the Stella language,[1] adding support for references, exceptions, and subtyping. More precisely, your implementation must support the following:

- all mandatory parts of Stage 1,
- sequential execution (`#sequencing`),
- references (`#references`),
- errors (`#panic`),
- exceptions with values (`#exceptions` and `#exception-type-declaration`),
- structural subtyping (`#structural-subtyping`),
- type casting (`#type-cast`),
- top and bottom types (`#top-type` , `#bottom-type`),
- disambiguation of ambiguous types using `Bot` (`#ambiguous-type-as-bottom`).

2.2 Implementation Requirements

The main goal of the project is to implement a *type checker*, a program that performs type checking for the model language Stella. You may reuse an existing parser and abstract syntax tree structure; however, the type checking algorithm itself and the auxiliary definitions must be implemented by each student *individually*.

The project may be implemented in any programming language, subject to prior approval by the instructor. However, it is recommended to use languages supported by the BNF Converter[2] or ANTLR,[3] since for these tools there already exists a ready-made grammar from which you can generate the basic project infrastructure.

2.2.1 Input and Output Format

The *type checker* must read a Stella program from the standard input (`stdin`) and write the result of the type checking to the standard output (`stdout`) and standard error (`stderr`).

If the input program does not contain type errors, the program must terminate with exit code `0`. Otherwise, it must terminate with a non-zero exit code.

If there are type errors, the first such error must be printed to the standard error stream (`stderr`). The error message must contain a human-readable description of the error, as well as a machine-readable error code.

Below is an example of a Stella program with a type error, followed by an example of an error message:

```
// a program in Stella
language core;

extend with
  #structural-subtyping,
  #references,
  #top-type,
  #unit-type,
  #type-ascriptions;

fn main(x : &Nat) -> Unit {
```

[1] https://fizruk.github.io/stella/
[2] https://bnfc.digitalgrammars.com
[3] https://www.antlr.org

```
  return x := (0 as Top)
}
```

```
// error message
ERROR_UNEXPECTED_SUBTYPE:
  expected a subtype of type
    Nat
  but found type
    Top
  for expression
    0 as Top
```

2.2.2 Lexical and Syntactic Analysis

For lexical and syntactic analysis, you are encouraged to use the existing Stella grammars together with parser generators BNFC[4] or ANTLR.[5]

BNFC supports code generation for Haskell, Agda, C, C++, Java (via ANTLR), and OCaml. Experimental backends exist for TypeScript and Dart. BNFC is a layer on top of other parser generators and additionally provides a well-structured abstract syntax tree and pretty-printing support.

ANTLR supports generation for Java, C#, Python 3, JavaScript, TypeScript, Go, C++, Swift, PHP, and Dart.

2.2.3 Requirements for the Type Checker

Your implementation of the *type checker* **must** support the following syntactic constructs of Stella (names of AST nodes are given in `typewriter` font):

1. all mandatory constructs of Stage 1;
2. for the `#sequencing` extension: `Sequence`
3. for the `#references` extension: `TypeRef`, `Ref`, `Deref`, `Assign`, `ConstMemory`
4. for the `#panic` extension: `Panic`
5. for the `#exceptions` and `#exception-type-declaration` extensions: `DeclExceptionType`, `Throw`, `TryWith`, `TryCatch`
6. for the `#structural-subtyping` extension: no new constructs (but the presence of the extension must be checked);
7. for the `#type-cast` extension: `TypeCast`
8. for the `#top-type` and `#bottom-type` extensions: `TypeTop`, `TypeBottom`
9. for the `#ambiguous-type-as-bottom` extension: no new constructs (but the presence of the extension must be checked);

When a type error occurs, the *type checker* must terminate with a non-zero exit code and print to the standard error stream a message containing a description of the problem and an error code.

For this assignment, you must use one of the following error codes:

1. the error codes of Stage 1;
2. `ERROR_EXCEPTION_TYPE_NOT_DECLARED` — the program uses exceptions but their type is not declared;
3. `ERROR_AMBIGUOUS_THROW_TYPE` — ambiguous type of a `throw` expression (`Throw`);

[4] https://bnfc.digitalgrammars.com
[5] https://www.antlr.org

4. `ERROR_AMBIGUOUS_REFERENCE_TYPE` — ambiguous type of a memory address (`ConstMemory`);
5. `ERROR_AMBIGUOUS_PANIC_TYPE` — ambiguous type of a panic (`Panic`);
6. `ERROR_NOT_A_REFERENCE` — attempt to dereference (`Deref`) or assign to (`Assign`) an expression that is not of reference type;
7. `ERROR_UNEXPECTED_MEMORY_ADDRESS` — a memory address (`ConstMemory`) is used where a type other than a reference type (`TypeRef`) is expected;
8. `ERROR_UNEXPECTED_REFERENCE` — a reference expression (`Ref`, i.e. `new(<expression>)`) is used where a non-reference type is expected;
9. `ERROR_UNEXPECTED_SUBTYPE` — the type of an expression is not a subtype of the expected type; this error must be raised only if none of the more specific errors above have been raised first.

Important! In this stage you **must** check the following extensions in the input Stella program and adjust the behaviour of the type checker accordingly:

- `#structural-subtyping` — subtyping is only enabled when this extension is used;
- `#ambiguous-type-as-bottom` — some type errors disappear when this extension is used.

Other extensions do not require an explicit check in your implementations, since they do not affect the type checking mechanism itself.

2.2.4 Optional Extensions and Error Codes

The following extensions may be implemented for extra credit:

1. `#open-variant-exceptions` : `DeclExceptionVariant`
2. `#try-cast-as` : `TryCastAs`
3. `#type-cast-patterns` : `PatternCastAs`

The following codes are relevant to Stage 2 features and may also be implemented for extra credit:

1. `ERROR_DUPLICATE_EXCEPTION_TYPE` — more than one exception type declaration (`#exception-type-declaration`) in the same scope (only one is allowed);
2. `ERROR_DUPLICATE_EXCEPTION_VARIANT` — duplicate exception variant label in open variant exception declarations (`#open-variant-exceptions`);
3. `ERROR_CONFLICTING_EXCEPTION_DECLARATIONS` — both `exception type` and `exception variant` declarations are present in the same scope (they cannot be mixed);
4. `ERROR_ILLEGAL_LOCAL_EXCEPTION_TYPE` — an exception type declaration (`#exception-type-declaration`) appears in an illegal (e.g. local) scope;
5. `ERROR_ILLEGAL_LOCAL_OPEN_VARIANT_EXCEPTION` — an open variant exception declaration (`#open-variant-exceptions`) appears in an illegal (e.g. local) scope.

2.3 Description of Stella Language Features

Stella is a programming language designed specifically for practicing the implementation of type checking algorithms. The core of the language is minimal in style and corresponds semantically to the simply typed λ-calculus with boolean and arithmetic expressions. On top of the core, Stella supports many extensions that allow you to gradually add syntactic and other features to the language.

Below we describe the language features that differ from those described in Stage 1.

2.3.1 Sequential execution (`#sequencing`)

Sequential execution is represented in Stella by expressions

```
<expression> ; <expression>
```

Semantics and typing rules for sequential execution follow the standard presentation [TaPL, §11.3].

2.3.2 References (`#references`)

References are represented in Stella by types `&<type>` and expressions

```
new(<expression>)            // create a reference
*<expression>                 // dereference
<expression> := <expression>  // assignment
<address>                     // reference as an explicit memory address
```

Semantics and typing rules for references follow the standard presentation [TaPL, §13].

Example of a well-typed program with references:

```
language core;
extend with #unit-type, #references, #let-bindings, #sequencing;

fn inc_ref(ref : &Nat) -> Unit {
  return
    ref := succ(*ref)
}

fn inc3(ref : &Nat) -> Nat {
  return
    inc_ref(ref);
    inc_ref(ref);
    inc_ref(ref);
    *ref
}

fn main(n : Nat) -> Nat {
  return let ref = new(n) in inc3(ref)
}
```

2.3.3 Errors (`#panic`)

Errors are represented in Stella by the expression

```
panic!          // (unrecoverable) error
```

Semantics and typing rules follow the standard presentation [TaPL, §14.1].

`panic!` errors cannot be caught by `try-with` or `try-catch` constructs.

Example of a well-typed program:

```
language core;
extend with #panic, #pairs, #fixpoint-combinator;

// decrement
fn dec(n : Nat) -> Nat {
  return Nat::rec(n, {0, 0},
    fn(k : Nat) {
      return fn(p : {Nat, Nat}) {
        return { succ(p.1), p.1 }
      }
    }).2
}

// subtraction
fn sub(n : Nat) -> fn(Nat) -> Nat {
  return fn(m : Nat) {
    return Nat::rec(m, n, fn(k : Nat) { return dec })
  }
}

// division (with explicit parameter for recursive call)
fn mkdiv(div : fn(Nat) -> fn(Nat) -> Nat) -> fn(Nat) -> fn(Nat) -> Nat {
  return fn(n : Nat) {
    return fn(m : Nat) {
      return if Nat::iszero(n) then 0 else
        succ(div(sub(n)(m))(m))
    }
  }
}

// division
fn div(n : Nat) -> fn(Nat) -> Nat {
  return fn(m : Nat) {
    return
      if Nat::iszero(m)
        then panic!        // ERROR: division by ZERO!
        else fix(mkdiv)(n)(m)
  }
}

fn main(n : Nat) -> Nat {
  return div(n)(n)
}
```

2.3.4 Exceptions (#exceptions)

Exceptions with values are represented in Stella by the expressions

```
// throw an exception
throw <expression>

// attempt to evaluate an expression with recovery
try { <expression> } with { <expression> }

// attempt to evaluate an expression with recovery;
// <pattern> binds variables to the exception value
// thrown during evaluation of the first expression
```

```
try { <expression> } catch { <pattern> => <expression> }
```

When the `#exception-type-declaration` extension is used, the type of values allowed in exceptions is given by the declaration:

```
exception type = <type>
```

When the `#open-variant-exceptions` extension is used, the type of values allowed in exceptions is a variant type, whose labels are given by declarations:

```
exception variant <label> : <type>
```

Semantics and typing rules for exceptions follow the standard presentation [TaPL, §14].
Example of a well-typed program:

```
language core;
extend with #exceptions, #exception-type-declaration;

exception type = Nat

fn fail(n : Nat) -> Bool {
  return throw(succ(0))
}

fn main(n : Nat) -> Bool {
  return try { fail(n) } catch { a => true }
}
```

2.3.5 Structural subtyping (`#structural-subtyping`)

Structural subtyping in Stella extends the type checker with the subsumption rule [TaPL, §15.1] and the subtype relation corresponding to the standard presentation [TaPL, §15].

2.3.6 Type casting (`#type-cast`)

Type casting in Stella is represented by the expression:

```
<expression> cast as <type>
```

Semantics and typing rules for type casting follow the standard presentation [TaPL, §15.5.1].
Example of a well-typed program:

```
language core;
extend with #type-cast, #pairs, #top-type, #structural-subtyping;

fn dup(x : Top) -> { Top, Top } {
  return { x, x }
}

fn main(n : Nat) -> Nat {
  return (dup(n) cast as {Nat, Nat}).1
}
```

2.3.7 Top and bottom types (`#top-type`, `#bottom-type`)

Top and bottom types in Stella are represented by the types

```
Top // top type
Bot // bottom type
```

Semantics and typing rules for top and bottom types follow the standard presentation [TaPL, §15.4].

2.3.8 Typing of ambiguous types (`#ambiguous-type-as-bottom`)

For expressions whose type is ambiguous, when the `#ambiguous-type-as-bottom` extension is enabled, the bottom type (`Bot`) is used instead of reporting a type error.

Example of a well-typed program:

```
language core;
extend with #ambiguous-type-as-bottom, #structural-subtyping, #sum-types;

fn main(n : Nat) -> Bool + Nat {
  return (fn (x : Nat) {
    return inr(x) // here the inferred type is the sum type Bot + Nat
  })(n)
}
```

2.3.9 Dynamic type checking (`#try-cast-as`, `#type-cast-patterns`)

Dynamic type checking is represented in Stella by the expression

```
try { <expression> } cast as <type>
  { <pattern> => <expression> }  // branch for successful cast
with
  { <expression> }               // branch for failed cast
```

Semantics and typing rules follow the standard presentation [TaPL, §15.5.1].

Dynamic type checking is also available in pattern matching:

```
<pattern> cast as <type>
```

Such a pattern matches successfully if the type cast succeeds. The pattern must then match the specified type.

References

[TaPL] Pierce, B. C., 2002. *Types and Programming Languages*. MIT Press.